

Java SOLID principles

What the five principles are, how to apply them, how they map to design patterns, and where they get overused. Definitions stay tight; the judgment and experience questions are how I would talk through them.

1. What is SOLID, who introduced it, and why does it matter?

SOLID is five object-oriented design principles for writing code that is easy to change and extend. The acronym was popularised by **Robert C. Martin (Uncle Bob)** in the early 2000s (he gathered principles that were already circulating). It matters because it targets the real problem in long-lived code: **change**. Software that is not designed for change becomes rigid (one change breaks many things), fragile, and hard to test. SOLID pushes you toward small responsibilities, extension over modification, and dependencies on abstractions, which keeps a codebase changeable.

Letter	Principle	One-line idea
S	Single Responsibility	A class should have one reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	A subtype must be usable wherever its base type is
I	Interface Segregation	Many small interfaces beat one fat one
D	Dependency Inversion	Depend on abstractions, not concretions

2. Are SOLID principles rules or guidelines, do you always follow them, and what are the trade-offs?

They are **guidelines, not rules**, and I treat them that way. A successful application absolutely can violate SOLID (plenty of profitable software does), because the principles serve maintainability, not correctness. So no, I do not follow them dogmatically. The **trade-off** is that every principle adds **indirection and abstraction**, which has a cost: more interfaces, more classes, more to read. Applied well that cost buys flexibility where you need it; applied blindly it buys **overengineering**, layers of abstraction around code that never changes. My rule is to apply a principle when there is a **real, likely axis of change**, and to leave simple stable code simple.

3. How do SOLID principles improve maintainability and testability?

Maintainability improves because small responsibilities and extension-over-modification mean a change touches **one place** instead of rippling everywhere, so the blast radius of any edit shrinks. **Testability** improves mostly through Dependency Inversion: when a class depends on an **interface** rather than a concrete dependency, you can **inject a fake or mock** in a unit test and test the class in isolation, without a real database or network. Small single-responsibility classes are also easier to test because each has **fewer paths** to cover.

4. Single Responsibility Principle (SRP).

A class should have **one reason to change**, meaning it does one job. The classic violation is a class that mixes concerns: an `Invoice` class that also **calculates tax, saves itself to the database, and formats a PDF**, so a change to any of those three reasons forces you to edit (and retest) the same class. The fix is to split it:

`Invoice` holds the data, a `TaxCalculator`, a `InvoiceRepository`, and a `PdfFormatter` each own one

job. A class **can have many methods and still satisfy SRP**, as long as they all serve the same single responsibility; SRP is about reasons to change, not method count. It improves maintainability because each concern lives in one place, so a change is localised.

5. Open/Closed Principle (OCP).

Software should be **open for extension but closed for modification**: you add new behaviour by **adding code, not changing tested code**. The classic violation is a growing `if/else` or `switch` on a type (`if (type == CREDIT) ... else if (type == PAYPAL) ...`), because every new type means editing that method and risking the existing branches. You refactor it by introducing an **abstraction**: a `PaymentMethod` interface (or abstract base) with each type as its own implementation, so a new payment type is a new class and the existing code is untouched. This is exactly where **interfaces and abstract classes** earn their place, and where patterns like **Strategy** and **Factory** help, they let you plug in new behaviour without reopening the old.

6. Liskov Substitution Principle (LSP).

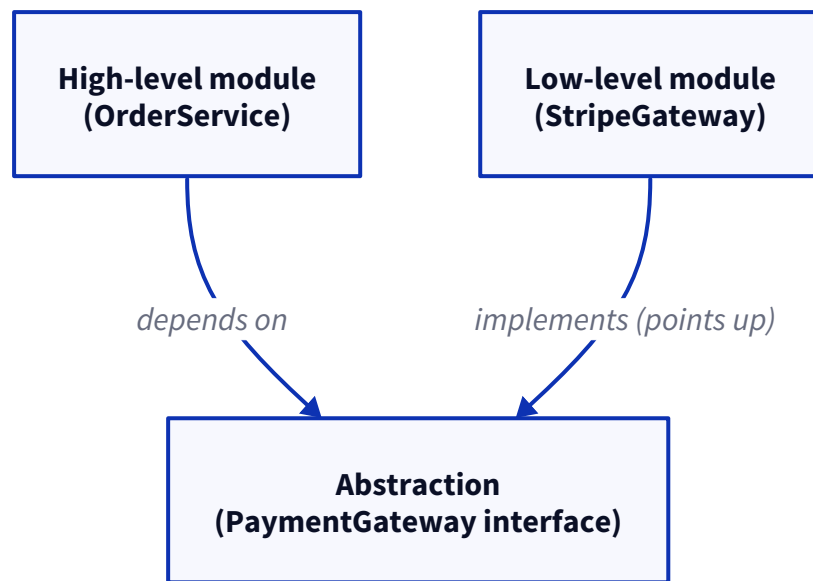
A **subtype must be usable anywhere its base type is expected**, without breaking behaviour. It matters because inheritance promises substitutability, and if a subclass violates the base class's contract, polymorphic code silently breaks. The **classic example** is `Square` extends `Rectangle`: a `rectangle` lets you set width and height independently, but a `square` forces them equal, so code that sets width to 5 and height to 4 on a `Rectangle` reference gets a wrong area when the object is really a `Square`. The subclass broke the parent's contract, so it is not a valid substitute. The lesson: inheritance must preserve behaviour, not just share fields.

7. Interface Segregation Principle (ISP).

Clients should not be forced to depend on methods they do not use, so prefer several small, focused interfaces over one fat one. The violation is a **"god interface"** like `Machine` with `print()`, `scan()`, `fax()`, `staple()`: a simple printer that only prints is still forced to implement `scan` and `fax` (usually by throwing "not supported"). The fix is to split it into `Printer`, `Scanner`, `Fax`, so each device implements only what it actually does. ISP is the interface-level cousin of SRP: keep the contract narrow.

8. Dependency Inversion Principle (DIP), and how it differs from Dependency Injection.

DIP says **high-level modules should not depend on low-level modules; both should depend on abstractions**. A **high-level module** holds the policy or business logic (an `OrderService`); a **low-level module** is a detail it uses (a `StripeGateway`, a `MySQLRepository`). Without DIP, the service directly `new`s the concrete gateway and is welded to it. With DIP, both the service and the gateway depend on a **`PaymentGateway` interface**, so you can swap the implementation freely. Interfaces are the whole mechanism: the abstraction in the middle is what both sides point at.



DIP is not the same as Dependency Injection. DIP is the **design principle** (depend on abstractions).

Dependency Injection is a **technique** for supplying those dependencies from outside (constructor injection, or a framework like Spring wiring them in) rather than creating them internally. DI is one common way to achieve DIP, but you could follow DIP without a DI framework, and you could inject a concrete class and still violate DIP. Principle v/s mechanism.

9. How do design patterns relate to SOLID?

Most design patterns exist to satisfy a SOLID principle; they are SOLID made concrete. A rough map:

Pattern	Mainly supports	Note
Strategy	OCP, DIP	Swap behaviour behind an interface without touching callers
Factory	OCP, DIP	Create objects without hard-coding the concrete class
Template Method	OCP	Fix the flow in a base, let subclasses extend the steps
Observer	OCP, SRP	Add new listeners without changing the subject
Dependency Injection	DIP	Supply abstractions from outside
Builder	SRP	Separates object construction from its representation

The caveat: a pattern **misused can violate SRP**. A Factory or a Builder that quietly grows business logic, or an Observer whose subject starts orchestrating its listeners, ends up with more than one responsibility. The pattern is a tool for a principle, not a guarantee of it.

10. How do interfaces help implement SOLID?

Interfaces are the connective tissue of most of SOLID. They enable **OCP** (add a new implementation instead of editing existing code), **LSP** (the interface defines the contract every implementation must honour), **ISP** (small interfaces are the whole point), and **DIP** (the abstraction both modules depend on). So a lot of "applying SOLID" in practice is "program to an interface, not a concrete class".

11. You are adding a new payment provider without modifying existing payment logic. Which SOLID principle applies?

Primarily the **Open/Closed Principle**: you are extending behaviour (a new provider) by adding a class, not changing the tested payment code. **Dependency Inversion** is right alongside it, because it only works if the existing code depends on a `PaymentProvider` **abstraction** rather than concrete providers. So OCP is the headline, DIP is what makes it possible.

12. How would you apply SOLID when exposing a public SDK?

For a public SDK the stakes are higher, because once people depend on your API you **cannot change it freely**, so I lean hardest on **Interface Segregation** and **Dependency Inversion**. I would expose **small, focused interfaces** so callers depend only on what they use, and keep concrete implementation classes **internal** so consumers program against the abstraction, not my internals. Open/Closed matters too: I would design extension points (interfaces, callbacks) so users can plug in behaviour without me changing the SDK, and so I can add features without breaking them. The guiding thought is that an SDK's public surface is a promise, so I keep it narrow and abstract. (*Adapt to an SDK you have shipped.*)

13. How has SOLID improved testability in your projects?

The concrete win is almost always **Dependency Inversion plus injection**. When a service takes its dependencies as **interfaces through the constructor**, I can unit-test it with **mocks or fakes**, no real database, network, or third-party call, so tests run fast and deterministically. Splitting fat classes by **SRP** helped too, because a small class with one job has few branches and is simple to cover. The code that was hardest to test was always the class that `new`ed its own dependencies and did five things; making it depend on abstractions was what unlocked the tests. (*Adapt to your own examples.*)

14. How do you teach juniors to apply SOLID without overengineering?

The way I frame it: SOLID is a response to **real, observed change**, not a checklist to apply upfront. I tell them to **start simple** and introduce an abstraction when a second reason to change actually appears (the second payment type, the second export format), not before, because the biggest mistake juniors make is **an interface for every class** and layers of abstraction around code that never varies. I point them at the **smell first** (a growing switch, a class doing three jobs) and let the principle be the fix for that smell. Rule of thumb I give them: duplication and a concrete pain point justify the abstraction; a hypothetical future does not.

15. How would you evaluate whether a large codebase follows SOLID without reviewing every class?

I would look for **signals and hotspots** rather than read everything. Version-control history shows the **files that change most often and together**, which points at SRP and OCP violations (one class edited for every kind of change). **God classes and god interfaces** show up by size (line count, method count, number of dependencies). A quick scan for **large switch / if-else on type** flags OCP problems, and **direct new of concrete services** instead of injected interfaces flags DIP problems. Static-analysis tools and dependency

graphs surface the worst offenders. So I target the churny, oversized, tightly-coupled parts, because that is where violations both exist and hurt. *(Adapt to your own tooling.)*

16. How would you review a pull request from a SOLID perspective, and what would you prioritise?

I prioritise the violations that will **cost the most later**. Top of the list: a class taking on a **second responsibility** (SRP), because that is how god classes start, and a **new if/else on type** where an abstraction belonged (OCP), because it will keep growing. Next, **concrete dependencies that should be injected interfaces** (DIP), since they block testing. Lower priority, and where I am careful not to over-push: I do **not** ask for an interface just for the sake of it, because forcing abstractions on simple stable code is its own smell. So I flag the changes that add real coupling or a real reason-to-change, and let genuinely simple code stay simple. *(Adapt to your own review style.)*

17. Common mistakes.

- **Applying SOLID blindly** without regard for the actual business requirements.
- **Creating excessive abstractions** and **overusing interfaces**, adding indirection with no payoff.
- **One interface for every class**, which is ISP misread as a mechanical rule.
- **Deep inheritance hierarchies** and **ignoring composition**, when composition would be simpler and more flexible.
- **Premature generalisation** and **overengineering for hypothetical future requirements**, paying today for change that may never come.
- **Misusing Dependency Injection** (injecting everything, huge constructors, DI where a plain **new** was fine).
- **Treating SOLID as strict rules** instead of design guidelines to apply with judgment.